Pattern visitor



Présenté par Loïc, Max et Mathias

Sommaire :

- Rappels sur SOLID
- introduction sur le GoF
- exemple et problématique
- modélisation avec et sans pattern Visiteur
- diagramme générique du pattern
- Présentation complète pattern visiteur
- diagramme classes et séquence
- tableau classes description
- limites et forces du visiteur
- lien entre visiteur et SOLID
- liens avec le pattern décorateur(diagramme de classes du déco + mise en relation)
- live-coding
- QCM

Les principes SOLID

SRP Single responsability principle

OCP Open closed principle

LSP Liskov’s substitution principle

ISP Interface Segregation principle

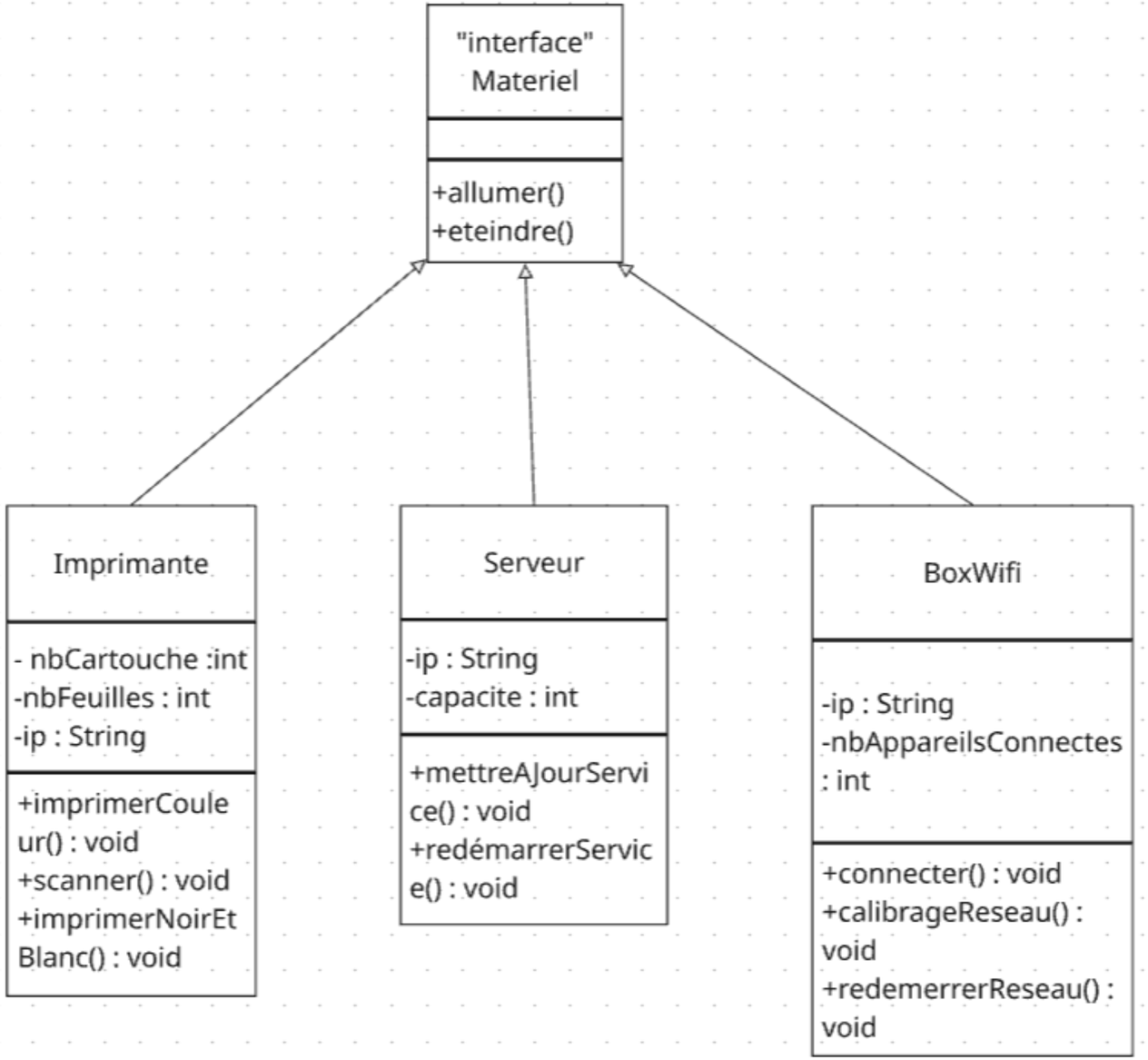
DIP Dependency inversion principle

Les patterns du GoF

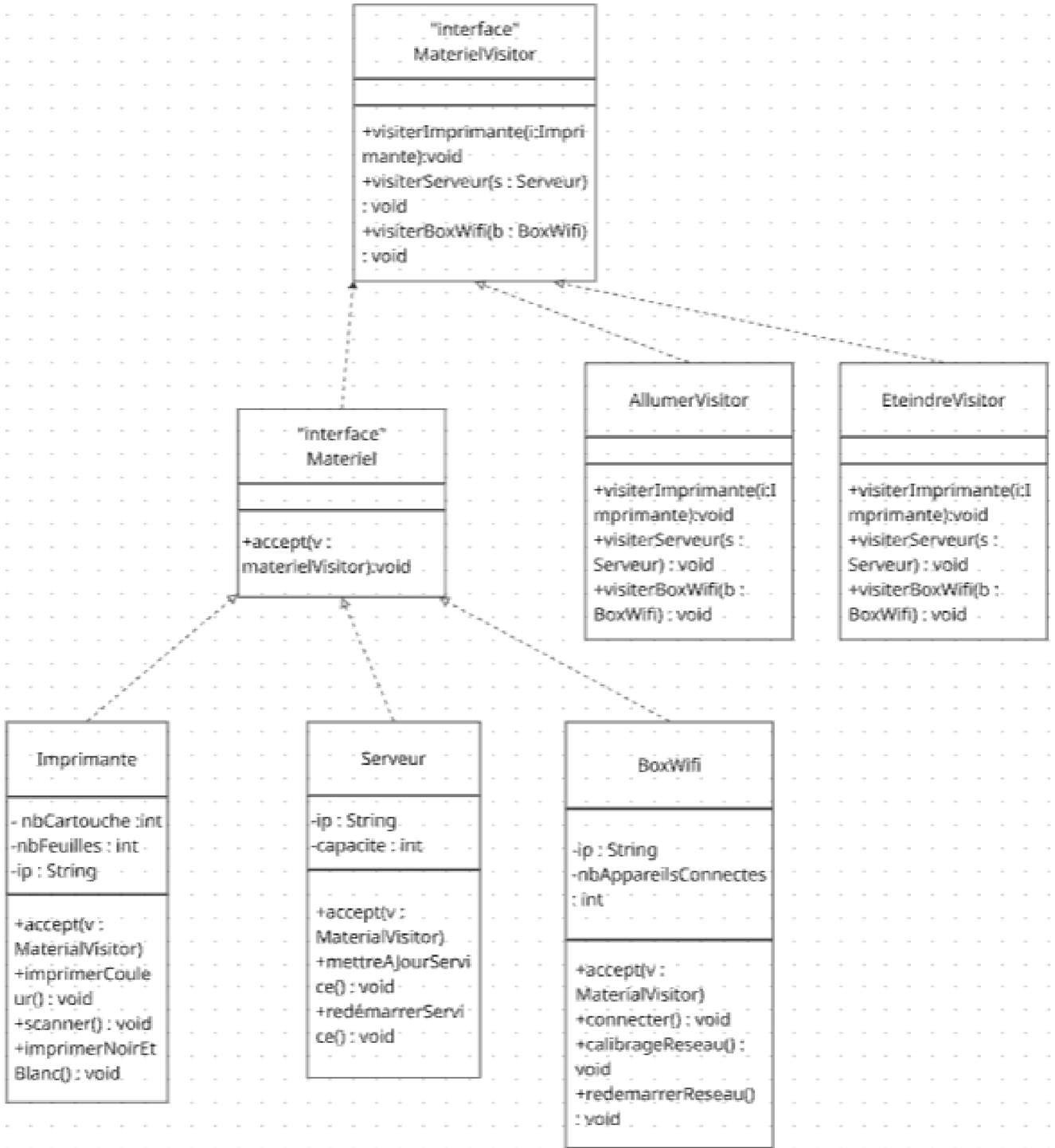
4 auteurs → 23 modèles

- Créationnels
- Structurels
- Comportementaux

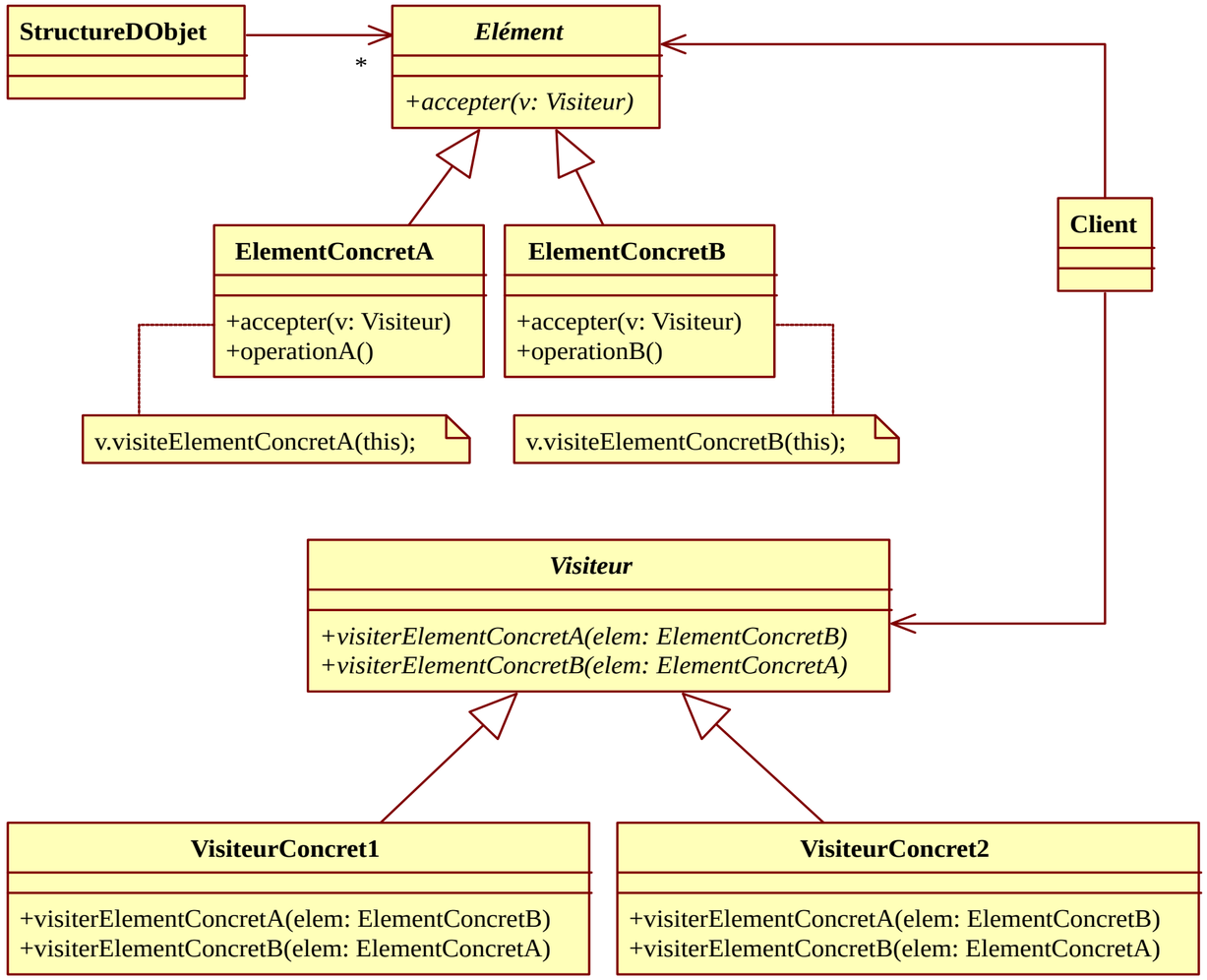
Exemple de solution sans visiteur:



Solution avec le pattern Visiteur :



Diagrame générique du pattern



Présentation du pattern Visiteur

Intention : ajouter de nouvelles opérations à une hiérarchie d'objets sans modifier ses classes

Principe : l'algorithme sort des éléments et va dans un objet Visiteur

Famille : pattern comportemental

Mécanisme :

1. element.accepter(visiteur)
2. visiteur.visiter(this)
3. le code exécuté dépend du type de l'élément ET du visiteur

Un autre exemple

```
abstract class Vehicule {
   protected String immatriculation;
   protected double dureeHeures;

   public abstract double calculerTarif();
   public String afficherTicket() { ... }
}

class Voiture extends Vehicule {
   private int emissionCO2;
   public double calculerTarif() { ... }
}

class Moto extends Vehicule {
   private int cylindree;
   public double calculerTarif() { ... }
}

class Camion extends Vehicule {
   private double poidsTonnes;
   public double calculerTarif() { ... }
}

class Parking {
   private List<Vehicule> vehicules;

   public void ajouter(Vehicule v) { ... }
   public double calculerTotal() { ... }
   public void afficherTickets() { ... }
}
```

```
abstract class Vehicule {
   public abstract void accept(Visiteur v);
}

class Voiture extends Vehicule {
   public void accept(Visiteur v) { v.visit(this); }
}

abstract class Visiteur {
   public abstract void visit(Voiture v);
   public abstract void visit(Moto m);
   public abstract void visit(Camion c);
}

class TarifVisiteur extends Visiteur {
   private double total;

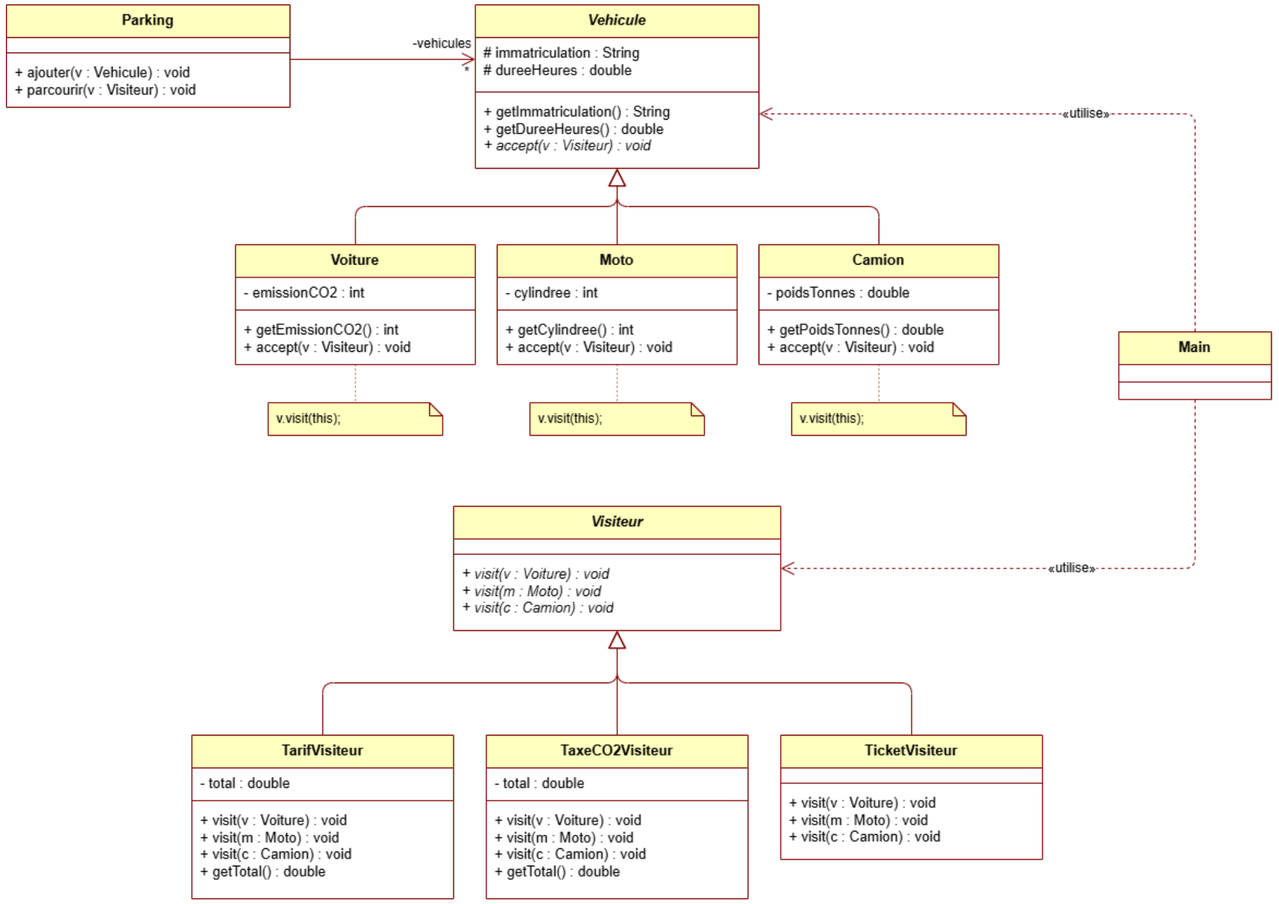
   public void visit(Voiture v) { total += v.getDureeHeures() * 2; }
}

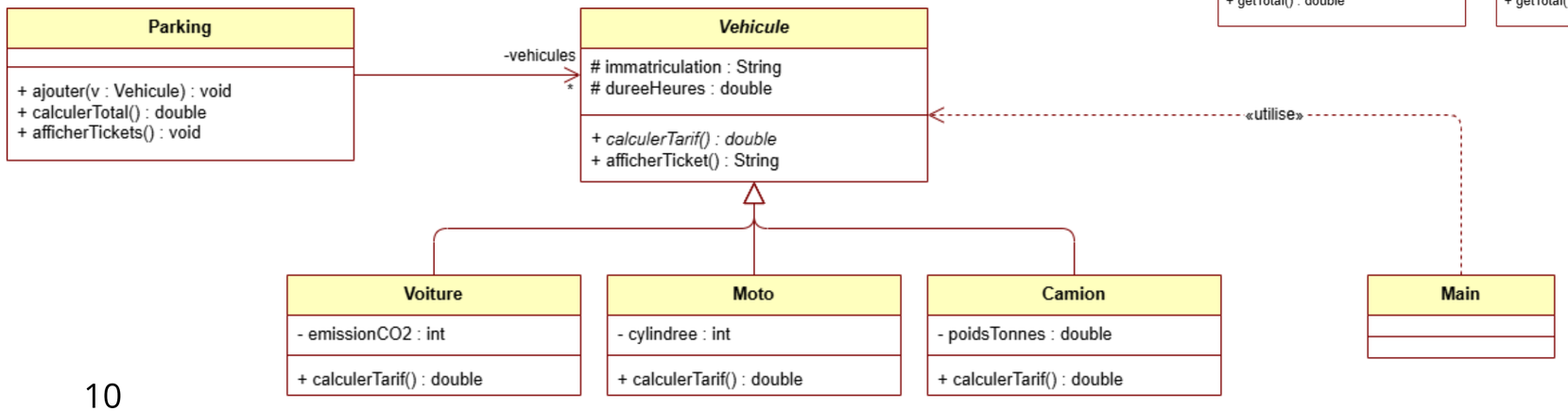
class Parking {
   private List<Vehicule> vehicules;

   public void parcourir(Visiteur visiteur) {
      for (Vehicule v : vehicules) v.accept(visiteur);
   }
}

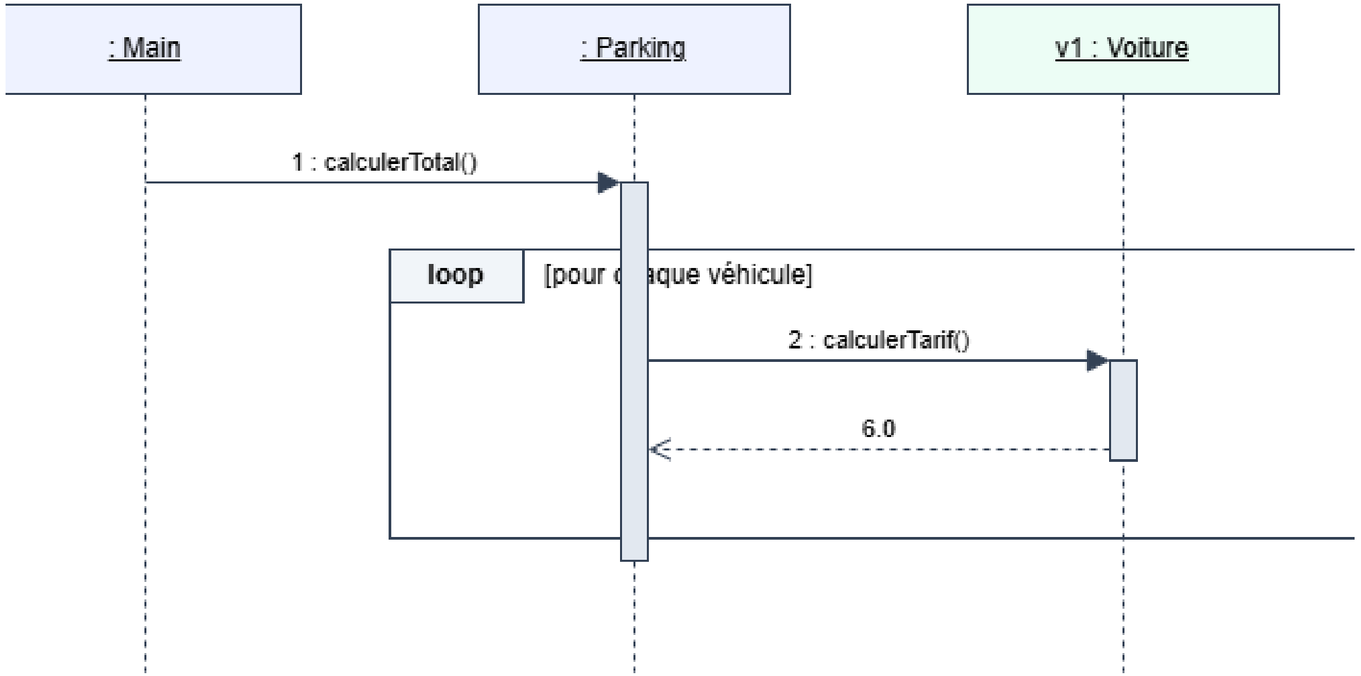
TarifVisiteur tarif = new TarifVisiteur();
parking.parcourir(tarif);
```

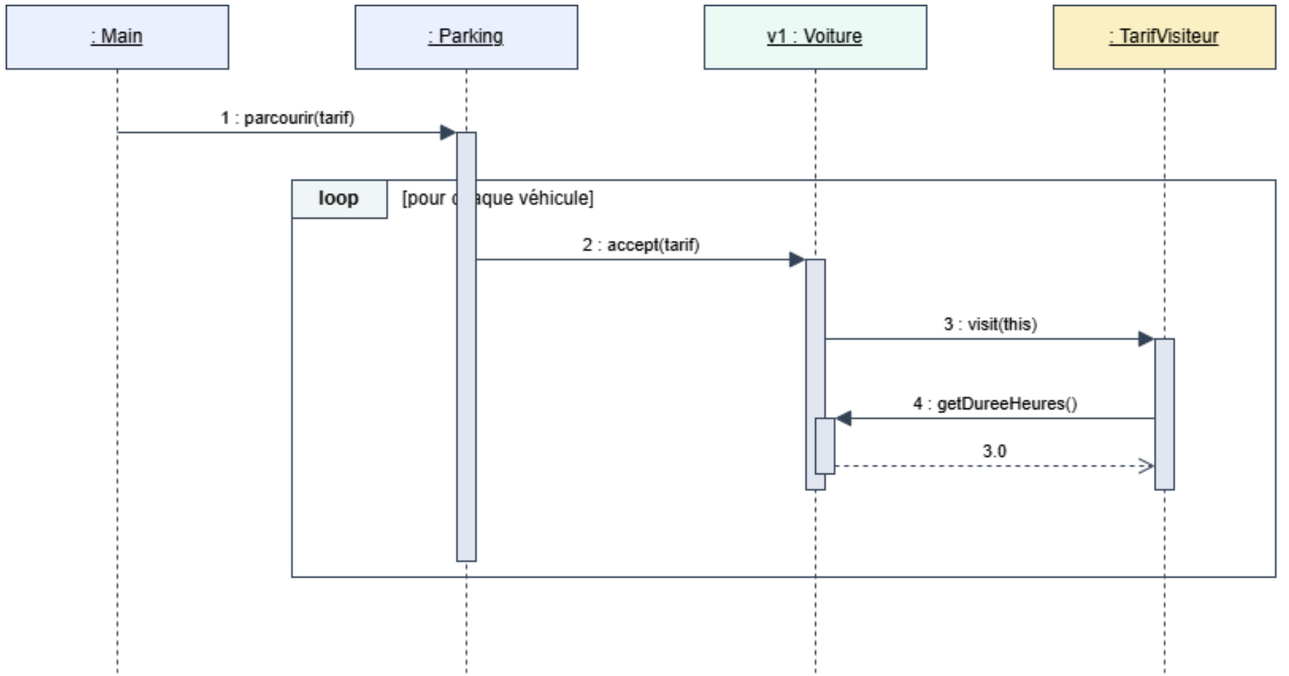
Comparaison des diagrammes de classe





Comparaison des diagrammes de séquence





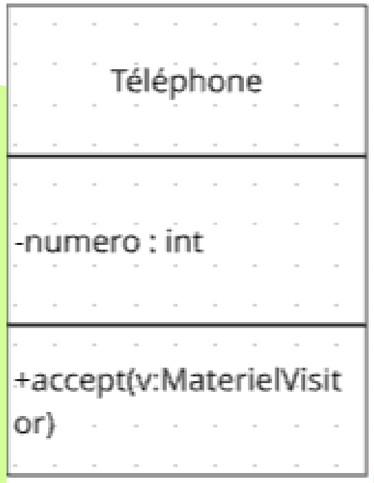
Rôle des classes

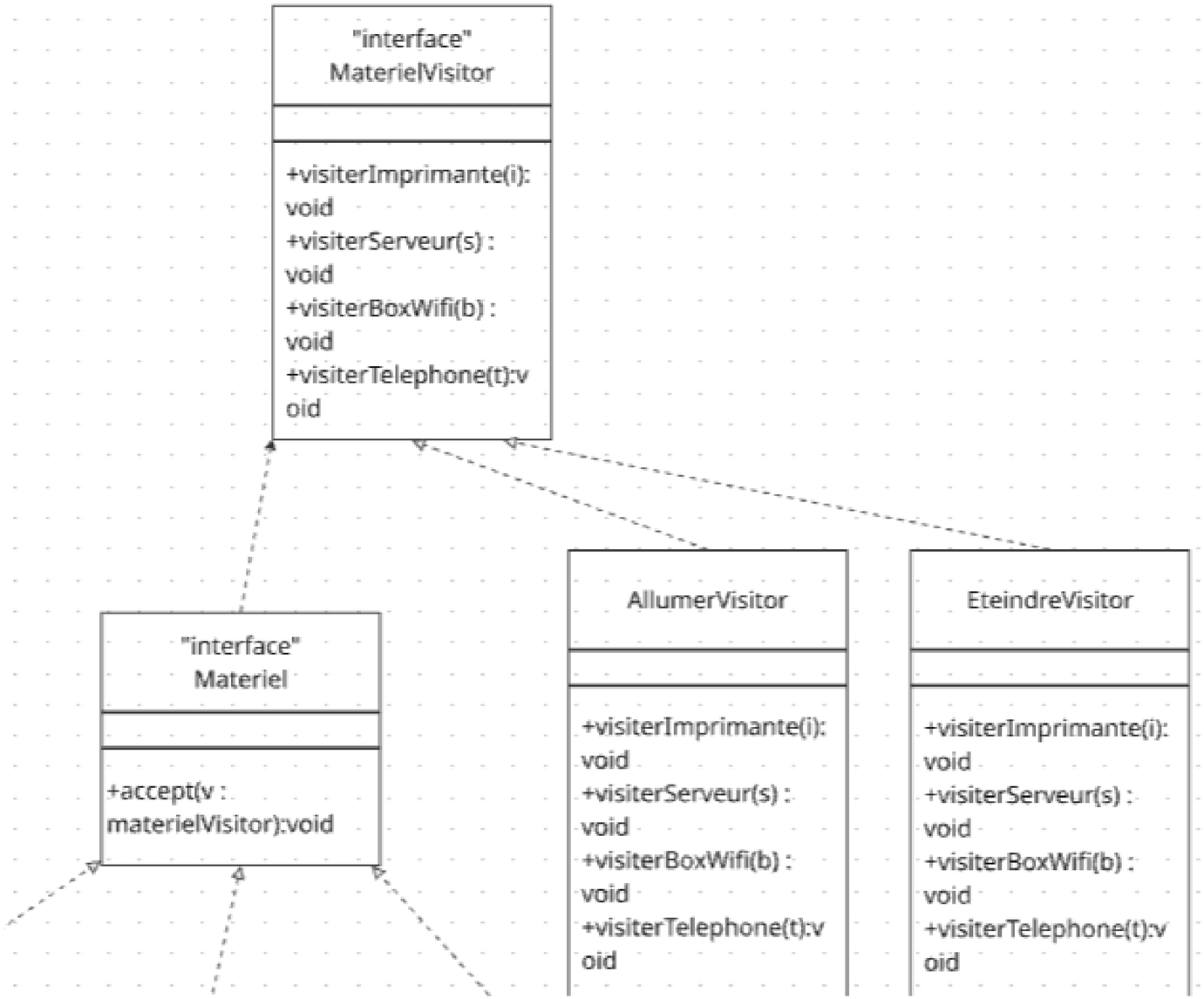
Nom diagramme générique	Nom dans notre diagramme
Element	Materiel
ConcreteElement	Imprimante Serveur BoxWifi
Visitor	MaterielVisitor
ConcreteVisitor	AllumerVisiteur EteindreVisiteur

Limites

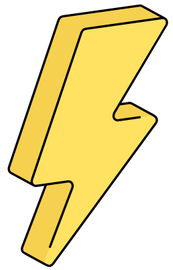


- Ajout de materiel

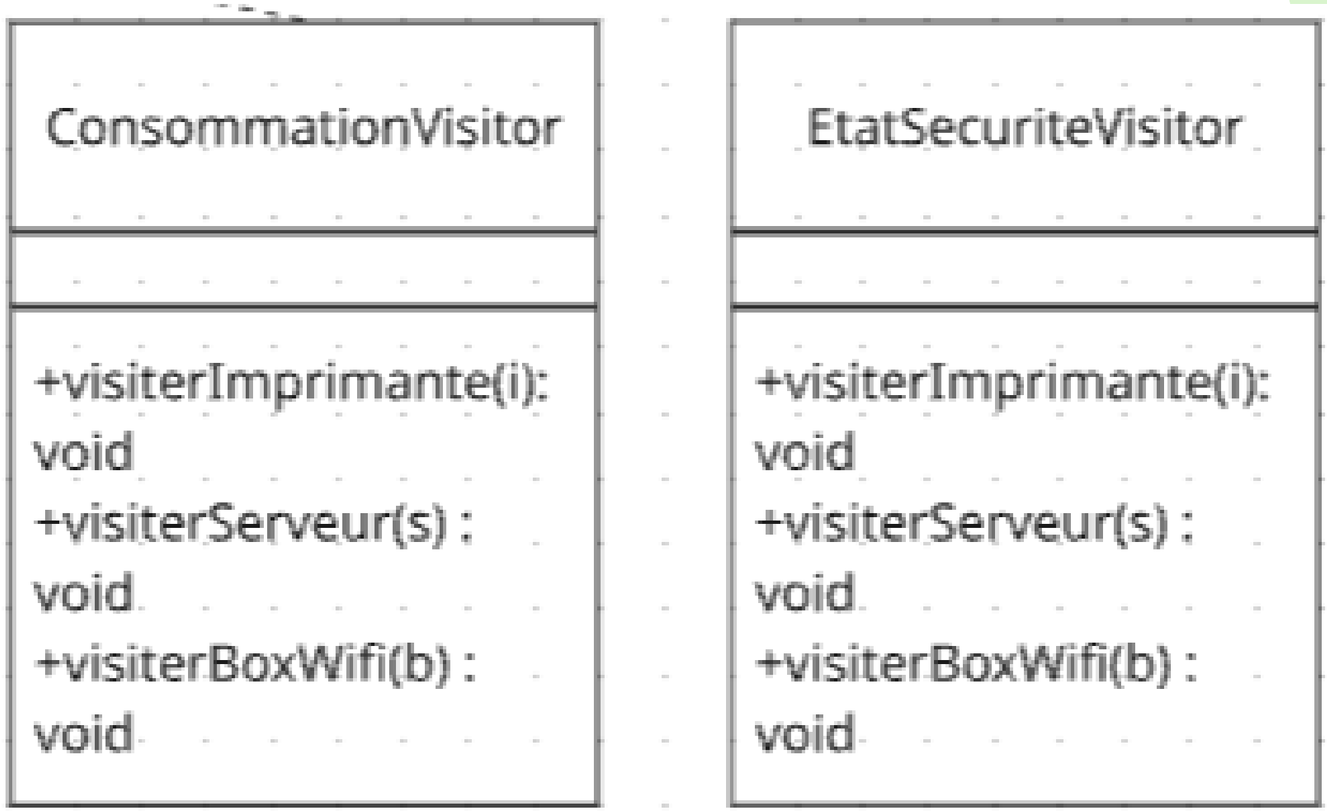




Force principale



- Ajout d’opération



Lien du pattern avec SOLID

SRP → ✓

OCP → ✓ / ✗

LSP → ✓

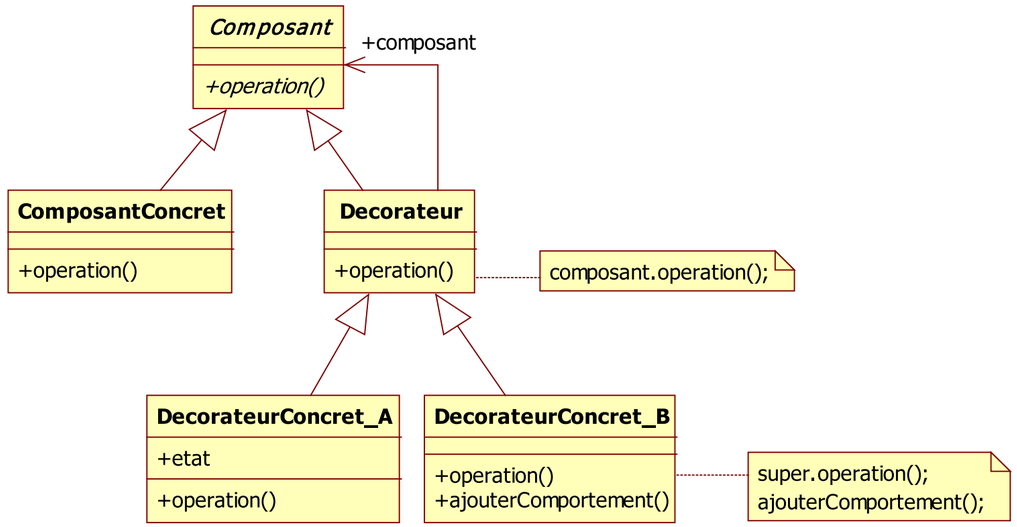
ISP → ✗

DIP → ✓ / ✗

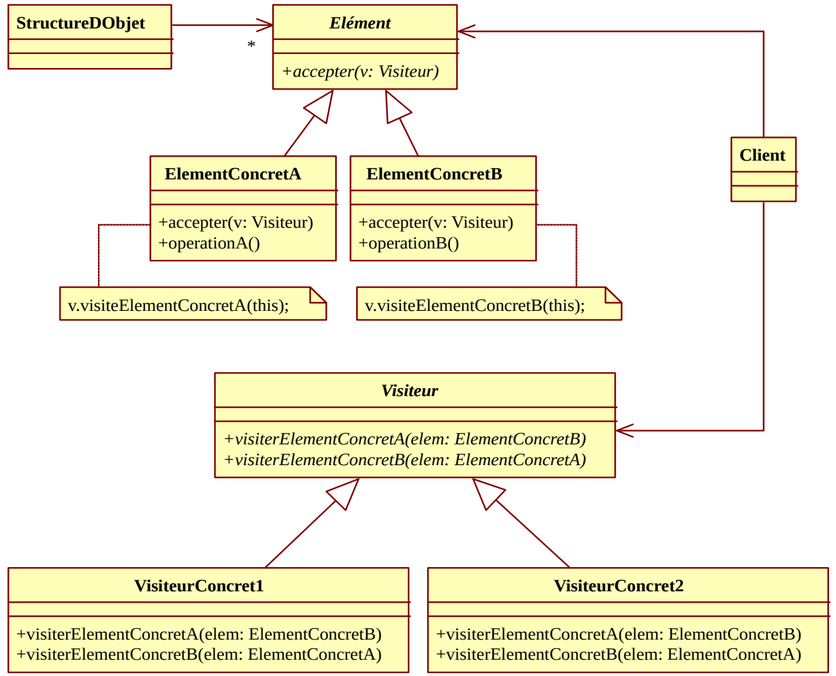
Comparaison avec le pattern décorateur

- Décorateur : un objet « enveloppe » un autre objet de la même interface et ajoute du comportement avant ou après avoir délégué. On empile les décorateurs à l'exécution.
- Visiteur : on sort une opération de la hiérarchie dans une classe à part, qui visite chaque élément via accept / visit (double dispatch).

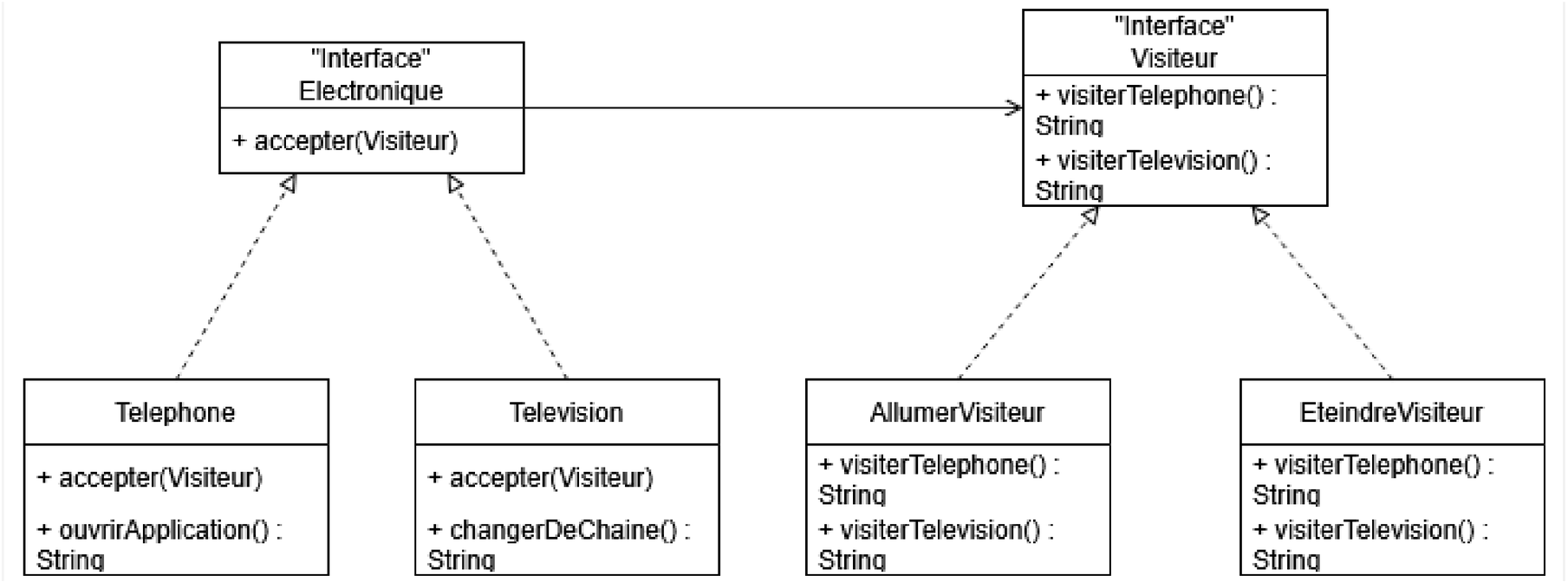
Décorateur



Visiteur



Et maintenant le live-coding

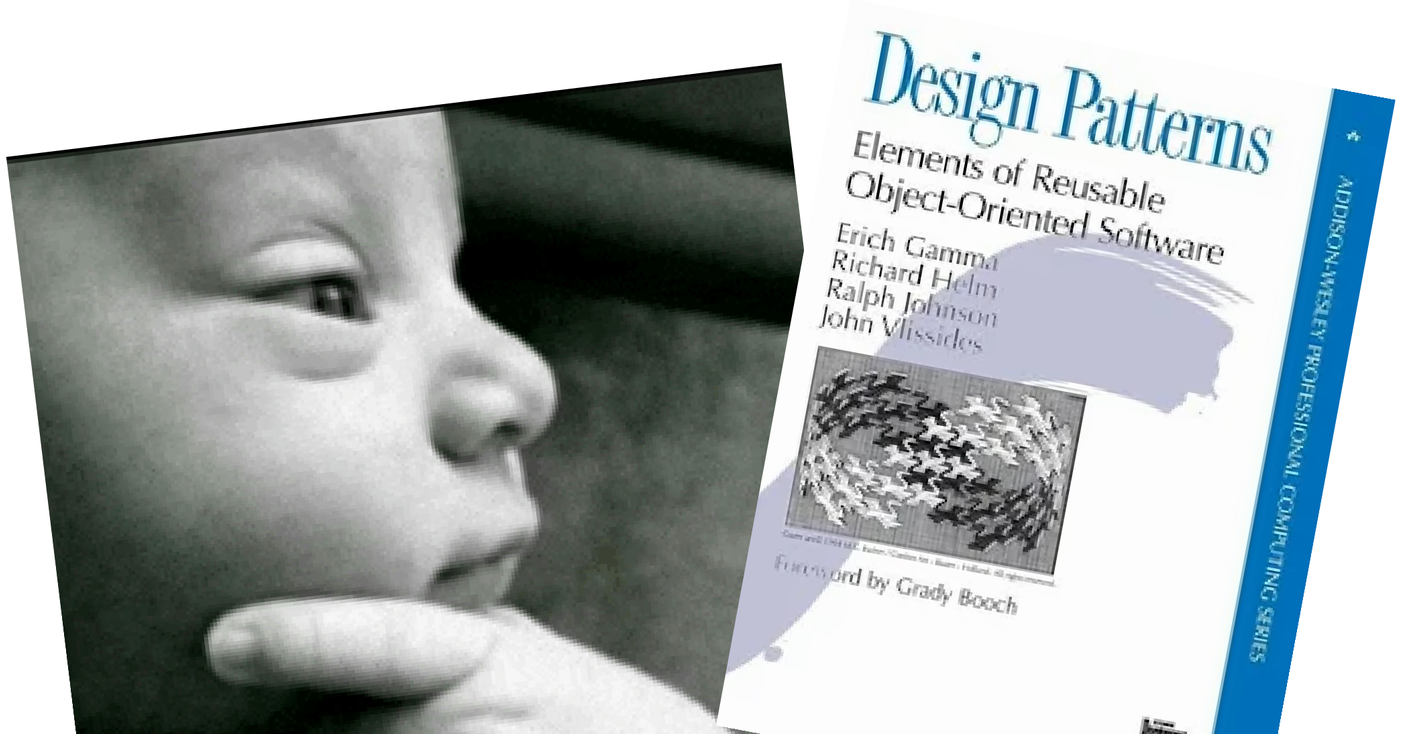


Bibliographie

bouhours.net http://www.goprod.bouhours.net/

alexsoyes.com https://alexsoyes.com/solid/#d-dependency-inversion-principle-dip

ionos.fr https://www.ionos.fr/digitalguide/sites-internet/developpement-web/visitor-pattern/



IAgraphie

Loïc

Trouver un exemple d’utilisation du pattern Visitor

Mathias

Trouver un exemple de cas d’utilisation et aide à la compréhension du pattern

Max

L’IA n’as pas été utilisée